

Explore to Plan

by Lorraine Gaudio

Lesson generated on October 19, 2025



BOISE STATE UNIVERSITY

Contents

0.1	Purpose	1
0.2	Column Names	1
0.3	Column Meanings	2
0.4	Data Types	3
0.5	Categorical Variables	4
0.6	Numeric Variables	4
0.7	Date/Time Columns	5
0.8	Data Quality Flags	6
0.9	Structure & Relational Needs	6
0.10	Display Planning	7
0.11	Submission Checklist	8

0.1 Purpose

In this activity you will closely inspect the columns in your chosen dataset and draft a short, practical plan for how you might cleanly name, type, and present those columns later in the project. This “look first, plan next” habit strengthens CLO2 (confidently problem-solving R tasks by narrating what’s happening, anticipating bugs, and comparing approaches) and advances CLO4 (readiness for the next R course through your Signature Assignment pipeline from raw data to a clear, narrated output).

MLO: Identify and justify specific future wrangling needs for your dataset. This includes naming conventions, data types/levels/units, and a simple table/graph intent. You will use inspection-only techniques and a brief written plan. You will **not** change the data today.

Below, each section has three parts:

1. ☐ What to look for.
2. ☐ Suggested functions to explore your data in R.
3. ☐ / ☐ Plan: what you should write for submission.

0.2 Column Names

In data science, teams usually pick a consistent naming style so columns are easy to read, type, and reuse.

0.2.1 Common styles:

snake_case: survey_date, total_amount (easy to type, common in R)

camelCase: surveyDate, totalAmount (common in many languages)

PascalCase: SurveyDate, TotalAmount (seen in some systems)

dot.case: survey.date (historical in R; can clash with some tooling)

(Avoid in R) kebab-case: survey-date (hyphens are subtraction)

0.2.2 ☒ What to look for

- **Spaces/punctuation** that make names hard to type.
- **Very long names** that slow you down or wrap across lines.
- **Inconsistent case** (mix of Age, age, AGE) across columns.
- **Ambiguous/duplicate** names (e.g., two different value columns).

0.2.3 ☒ Suggested functions

- `names()` or `colnames()` to list all column names.
- `ncol()` to sense how many columns you're dealing with.

What to notice in the output: scan for spaces, unusual symbols, repeated or near-duplicate names, inconsistent capitalization, and names that are much longer than the rest.

0.2.4 ☒☒ Plan

List the specific column names you plan to improve. Record the original column name(s) and the proposed name change. State why you will make this change.

Example: I will change “Resp Age” to `age_years` to remove the space, and clarify the units”.

0.3 Column Meanings

A mini data dictionary is a compact reference that defines what each important column means and how to read it. It's created as you inspect the data and is formatted as short entries (bullets or a simple 3-5 column table). Each entry typically includes: name, one-sentence description, role (predictor/outcome/ID/other), and unit or allowed values.

0.3.1 ☒ What to look for

- **Unclear purpose** (“What does val1 store?”).
- **Cryptic abbreviations** that need unpacking later.
- **Catch-all text columns** with mixed content that might need splitting.

0.3.2 ☒ Suggested functions:

- Use `head()` and `str()` to scan types while you think about meaning.

What to notice in the output: sample values that reveal what a column really stores, and whether the type matches your intuition about its meaning.

0.3.3 ☒☒ Plan

Identify the 1–3 most important columns for your project. For each, record:

| Name | Role (predictor/outcome/ID/other) | One-sentence description | Unit or allowed values (if known). |

0.4 Data Types

Getting data types right ensures R behaves as expected (e.g., plotting numbers as numbers, sorting categories in a sensible order).

Intended vs. actual: what you want the type to be for analysis versus what it currently appears to be in your data.

0.4.1 ☒ What to look for:

- **Numbers stored as text** (won’t summarize/plot numerically).
- **IDs stored as numbers** (R may treat them as quantities instead of labels).
- **Dates stored as text** (hard to sort or group by time).
- **Yes/No** as free text with inconsistent capitalization/spelling.

0.4.2 ☒ Suggested functions

- `class()` and `typeof()` to see how R currently treats a column.
- `unique()` to spot unexpected values that hint at the true type.

What to notice in the output: mismatches between the values you see and the type R reports (e.g., a “numeric-looking” column that’s actually “character”).

0.4.3 Plan

In a short paragraph, name 1-2 columns where the type matters for your analysis. For each, state the current type you observe, the intended type you want later (numeric, factor, date, etc.), and why that change will help (e.g., enables numeric summaries, proper date sorting, or category ordering).

0.5 Categorical Variables

When a column represents **categories** (like region or treatment_group), assigning **levels** (the set of allowed labels) and a **level order** can make tables/plots clear and consistent. Thoughtful labeling avoids typos/synonyms and level ordering controls how categories appear in summaries and graphs.

0.5.1 What to look for

- **Inconsistent spellings** (e.g., Female, female, F).
- **Extra spaces or synonyms** (NY vs New York).
- **Too many tiny categories** that may need grouping later.
- **Meaningful order** you'll want to enforce (e.g., Small < Medium < Large).

0.5.2 Suggested functions

- `unique()` to see the set of observed labels.
- `table()` to glimpse how often each label appears.
- `levels()` to check or discuss factor levels (if applicable in your data).

What to notice in the output: near-duplicate labels, rare categories that could be combined, and the “natural” order you’d prefer for displays.

0.5.3 Plan

Choose one categorical column. Write a brief note that includes: example observed labels, your canonical label set (the exact labels you want to use later), and the planned order (if an ordered category makes sense) with one sentence on why that order helps interpretation.

0.6 Numeric Variables

For numeric columns, clarity about units and scale prevents confusion (meters vs. feet, seconds vs. minutes). Sometimes it's helpful to plan a derived measure (like a rate or percentage) that communicates more directly than raw counts.

0.6.1 ☒ What to look for

- **Missing or mixed units** (document the current unit).
- **Suspicious magnitudes** that hint at a unit mismatch.
- **Opportunities for a clearer measure**, such as a percent, rate, or difference you might compute later.

0.6.2 ☒ Suggested functions

- Use `min()`, `max()`, `range()`, `summary()` to get a quick sense of magnitude and spread.

What to notice in the output: whether values align with the unit you think they're in, and whether a transformed/derived measure would make stories clearer.

0.6.3 ☒☒ Plan

Pick **one numeric column**. Record the current unit, the desired unit (if you'd convert later), and one derived field idea (give it a specific name and a plain-English formula, e.g., “`conversion_rate_pct` = conversions divided by visitors, expressed as a percent”), plus one sentence on why it would clarify your analysis.

0.7 Date/Time Columns

Dates and times need to be **parsed** into a proper date/time type so you can sort and group by time. It also helps to decide the **granularity** (day, week, month, quarter) you'll analyze so your tables/plots match the story you want to tell.

0.7.1 ☒ What to look for

- **Multiple formats** (e.g., 2024-01-05 vs 01/05/24).
- **Text dates** that haven't been parsed yet.
- **Time zones** that might matter for comparisons.
- The **time unit** you'll likely analyze by (day/week/month).

0.7.2 ☒ Suggested functions

`head()` to sample raw date/time strings.

`as.Date()` (referenced for later parsing)

What to notice in the output: inconsistencies in how dates are recorded and examples that show what format you'll want to standardize to.

0.7.3 ☒☒ Plan

If you have a date/time column, write a short note with raw examples you see, your intended standard format (describe it, e.g., “YYYY-MM-DD”), and the analysis grain you expect to use later (e.g., month-year), with one sentence on why that grain fits your question.

0.8 Data Quality Flags

Data often contains values you’ll want to flag or define rules for later (you’re not filtering today). Writing these rules now saves time when you clean the data.

0.8.1 ☒ What to look for

- **Impossible values** (e.g., negative ages).
- **Sentinel codes** standing in for missing (e.g., -999, 99).
- **Out-of-range** numbers for your context.
- **Duplicates** or near-duplicates.
- **Blank strings** that should be treated as missing.

0.8.2 ☒ Suggested functions

- `anyNA()`, `is.na()` to think about missingness.
- `nrow()` for quick comparisons (before/after a check).
- `table()` to surface odd codes or rare values.

What to notice in the output: evidence that certain values are actually placeholders or errors you’ll need to handle consistently later.

0.8.3 ☒☒ Plan

Write 2–3 future rules you plan to apply, each with a short rationale.

0.9 Structure & Relational Needs

Sometimes the data’s **shape** (wide vs. long) or **relationships** (needing to join multiple tables) don’t match your question. Noting this now helps you plan future steps.

0.9.1 ☒ What to look for

- **Wide vs. long layout** that doesn't align with how you want to compare or plot.
- **Composite columns** that pack multiple fields into one (e.g., "City, State").
- **Separate files** you'll likely need to join (e.g., a lookup table of codes → labels).
- **Missing unique keys** needed to join tables reliably.

0.9.2 ☒ Suggested functions

- `ncol()` / `nrow()` to sense the table's shape.
- `head()` to eyeball how columns are arranged.

What to notice in the output: repeated patterns across columns (wide form), or single columns carrying multiple pieces of information.

0.9.3 ☒☒ Plan

If structure matters for your project, describe one reshape or join you anticipate. Name the columns involved and explain in one sentence why this change will make your eventual table or graph clearer (e.g., "Join codes to human-readable labels for presentation").

0.10 Display Planning

Before you ever plot, decide on a minimal, meaningful table and a first graph. This clarifies what needs renaming, leveling, units, or structural changes later.

0.10.1 ☒ What to look for

A minimal, meaningful table: which columns would you show, in what order, and would you sort or total anything?

A first graph idea: which column maps to x, which to y, and whether you need a group/color variable; if categorical, what order should those categories appear in?

0.10.2 ☒☒ Plan

Write one table idea and one graph idea for your project. For the table, list the columns to include and any sort/total you want. For the graph, specify x, y, and optional group/color, and (if categorical) the intended category order. Add one sentence on why each display would help your audience understand your question.

0.11 Submission Checklist

- ☐ Posted HTML (or .Rmd) to Module 8 Class Activity 8 by 11:59 pm. (In-class activities should not become homework. Ideally, you submit your work as far as you get by the end of class.)
- ☐ Section titled “Data Overview”
- ☐ At least four subsections (Column Names, Column Meanings, Data Types, Categorical Variables, Numeric Variables, Date/Time Columns, Data Quality Flags, Structure Relational Needs).
- ☐ Exploratory Code for each subsection.
- ☐ Comment on Plan / Metacognition for subsections.